Companion to Bird & de Moor, Algebra of Programming

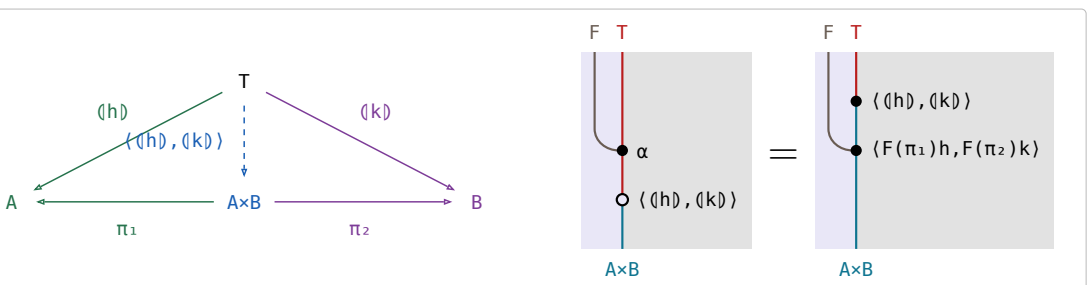
§3. Applications

§3.1. Banana-split

(3.1a)

	definition	type	note
1	`listr A::=nil|cons(A,listr A)  list(A) = [A]`	$\mathcal{A}\to\mathcal{A}$	The cons-lists over `A`, the datatype every row below folds.
2	`sum = (|[zero,plus]|)`	`[ℕ]→ℕ`	`plus(a,b)=a+b`.
3	`length = (|[zero,π₂ succ]|)`	`[A]→ℕ`	`π₂` drops the head and keeps the count of the tail, `succ` adds one for the head.
4	`average ≜ ⟨sum,length⟩ div`	`[ℕ]→ℕ`	`div(m,n)=m/n`, with `div(0,0)=0` so `average` is total. Traverses the list twice.
5	banana-split law `⟨(|h|),(|k|)⟩ = (|⟨F(π₁) h,F(π₂) k⟩|)`	`T→A×B`	Any fork of folds is a single fold, hence one traversal — `F` the base functor.
6	what it reduces to `α ⟨(|h|),(|k|)⟩ = F(⟨(|h|),(|k|)⟩) ⟨F(π₁) h, F(π₂) k⟩`	`FT→A×B`	All that ≜ `reduce` leaves to check: the fork satisfies the defining equation.
7	the instance `⟨sum,length⟩ = (|[zeros,pluss]|)`	`[ℕ]→ℕ×ℕ`	`pluss(a,(b,n))=(a+b,n+1)`, so `average` runs in one pass.
8	`preds n=[n,n−1,…,1]`	`ℕ↛[ℕ]`	apply the earlier special case to write `preds` as `(|k|)π₁`.

(3.1b)

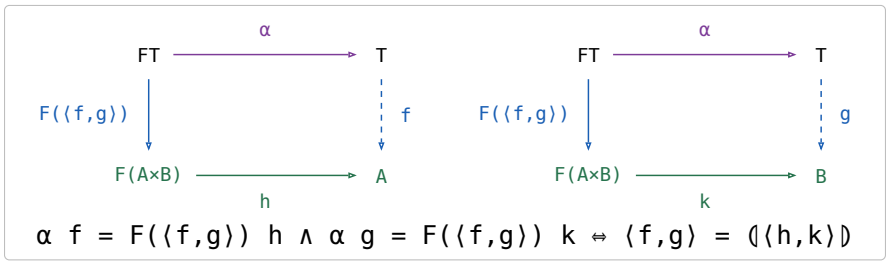


`⟨(|h|),(|k|)⟩ = (|⟨F(π₁) h,F(π₂) k⟩|)` banana split

§3.1.1. Fokkinga's mutual recursion theorem

- `F-Alg(𝒜)` — the algebras and their homomorphisms — has binary products whenever `𝒜` does, and the forgetful `U : F-Alg(𝒜)→𝒜` **creates** them: the product of `h : FA→A` and `k : FB→B` is carried by `A×B`, with structure `⟨F(π₁)h,F(π₂)k⟩ : F(A×B)→A×B`.
- `π₁` and `π₂` are homomorphisms out of it, and `⟨p,q⟩ : X→A×B` is a homomorphism **iff** `p` and `q` both are — substitute the structure and compare the two forks component by component.
- The banana-split law of `⟨(|h|),(|k|)⟩ = (|⟨F(π₁) h,F(π₂) k⟩|)` IS that product's universal property read at the initial algebra: `(|h|)` and `(|k|)` are the unique homomorphisms to `(A,h)` and `(B,k)`, so their fork is the unique homomorphism into the product, which is `(|⟨F(π₁)h,F(π₂)k⟩|)`.
- Fokkinga's theorem is **strictly more general** and is not that product: in mutual recursion the two arrows leave `F(A×B)`, not `FA` and `FB`, so each sees BOTH components. The product is the case where they factor as `F(π₁)h` and `F(π₂)k`. What it says: **any algebra on a product carrier is folded by a fork**.

(3.1.1a)



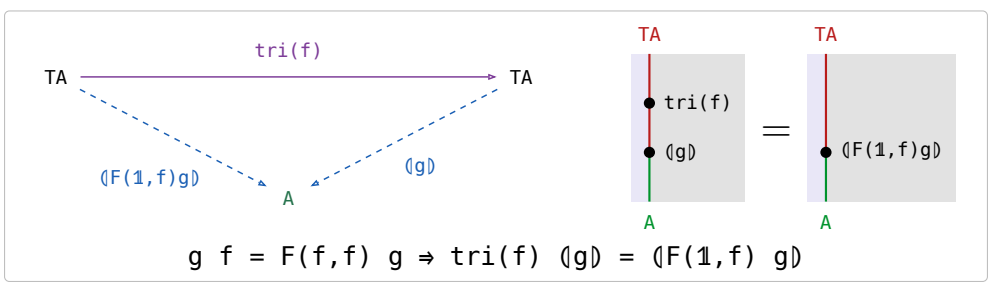
§3.2. Ruby triangles and Horner's rule

(3.2a)

	stage	definition
1	informally	`tri(f)[a₀,a₁,…,aᵢ,…,aₙ]=[a₀,f(a₁),…,fⁱ(aᵢ),…,fⁿ(aₙ)]`
2	for cons-lists	`tri(f)=⦇[nil,(𝟙×listr(f)) cons]⦈`
3	with the base functor named	`F(A,B)=1+A×B`, `α=[nil,cons]`, so `tri(f)=⦇F(𝟙,listr(f))α⦈`
4	abstractly	`F` a bifunctor with initial type `(α,T)`: `tri(f) = ⦇F(𝟙,T(f)) α⦈`

For the definition to make sense `f : A→A` is required, and then `tri(f) : TA→TA`.

(3.2b)



Horner's rule is Horner's rule, generalised from cons-lists to any initial type. Fusion reduces it to `F(𝟙,T(f))α⦇g⦈=F(𝟙,⦇g⦈)F(𝟙,f)g`.

§3.2.1. Depth of a tree

(3.2.1a)

1	the datatype	`tree A::=tip(A)|bin(tree A,tree A)`, base functor `F(A,B)=A+B×B`, initial type `([tip,bin],tree)`
2	the fold	`(|[g,h]|)` is the unique `f` with `f(tip(a))=g(a)` and `f(bin(x,y))=h(f(x),f(y))`
3	`tree(f)`	`tree(f)=(|F(f,1) [tip,bin]|)`; pointwise `tree(f)(tip(a))=tip(f(a))` and `tree(f)(bin(x,y))=bin(tree(f)(x),tree(f)(y))`
4	`max`	`max=(|[1,bmax]|)`, where `bmax(a,b)` is the larger of `a` and `b`
5	`depths`	`depths=tree(zero) tri(succ)` — replaces every tip by its depth in the tree, `zero` the constant function returning 0 and `succ` the successor
6	`depth`	`depth=depths max`

§3.3. The TeX problem

§3.4. Conditions and conditionals

§3.5. Concatenation and currying

Theorem 3.1 `If φ is natural, i.e. G(h×id)·φ=φ·(F(h)×id), then h·G(f)·φ=f ⇔ f=apply·((h·G(apply)·φ)×id)` The unique solution of a two-argument recursion `f(a,x)=h(G(f)(a,x))` built from a natural `φ` is `apply` composed with a one-argument fold, which is how `cat` — not itself a fold — is derived from one.